

Merged Modules and Tooling

Document: [P1156R0](#)
Audience: EWG
Authors: Boris Kolpackov
Reply-To: boris@codesynthesis.com
Date: 2018-10-04

Note

A draft of this paper was discussed at the Bellevue (ad hoc) meeting ([P1136R0](#)) as well as referenced by another paper ([P1180R0](#)). The contents of that draft are therefore preserved unchanged in this final paper with post-meeting notes added at the end of each section and marked with the BELLEVUE keyword.

Contents

- 1 [Abstract](#)
- 2 [Global Fragment Restrictions](#)
- 3 [Non-Modular Code](#)
- 4 [Preamble End](#)
- 5 [Module Partitions](#)
- 6 [Acknowledgments](#)

1 Abstract

This paper describes a number of tooling-related issues that we have identified with the merged modules proposal ([P1103R0](#)). Summary of the proposed changes:

- Remove `#include` s-only restriction on the global module fragment.
- Add import preamble requirement to non-module translation units.
- Add explicit preamble end marker.

2 Global Fragment Restrictions

[P1103R0 Section 2.3.1](#) Clause 2 states:

"Only `#include` s are permitted to appear in the global module fragment, but there are no special restrictions on the contents of the `#include` d file."

The rationale for this restriction given at the Rapperswill meeting is to allow simple module-aware tools without the need for preprocessing or elaborate parsing. It was also acknowledged that such tools won't be able to handle all valid module translation units since both module declarations and import declarations can be `#include` ed.

While not handling "exotic" translation units like this may be an acceptable trade off, such simple tools will also fail to handle (or, more likely, mishandle) translation units that use `#if` for conditional importation. And we believe this will be common in real-world code, for example:

```
module foo;

#ifdef EXTRA
import bar;
#endif

...
```

Furthermore, this restriction prevents useful practices, most notably, the ability to forward-declare in the global module fragment, for example:

```
module;  
  
//#include "heavy.h" // Expensive.  
class heavy;          // Illegal.  
  
module foo;  
...  

```

Additionally, the complexity of deciding when to stop scanning for module-related declarations (see [Preamble End](#)) will most likely result in compiler implementations providing support for extracting information from the module preamble (see, for example, GCC's `-fmodule-preamble`). With this support, simple tools should be able to achieve greater reliability without significant extra complexity.

As a result, because this restriction only offers false hope of simplicity while preventing established and useful practices, we propose that it be removed.

BELLEVUE: It was suggested during the meeting that instead of relaxing the global module fragment restrictions, it should be removed entirely in favor of using legacy header modules. This, however, would make it impossible to include into modular code headers that are not sufficiently well-behaved to be represented as legacy modules.

3 Non-Modular Code

[P1103R0 Section 2.3.3](#) Clause 1 states:

"Modules and legacy header units can be imported into non-modular code. Such imports can appear anywhere, and are not restricted to a preamble."

Furthermore, from [P1103R0 Section 19.3](#) Clause 1 it follows that in such non-module units macros exported by a legacy header module are visible immediately after the import declaration (as opposed to at the end of the preamble) and therefore can affect subsequent importations.

As discussed in detail in [P1052R0 Section 3](#), this "relaxed" model for non-module translation units will significantly complicate module dependency extraction by build

systems and other tools. Briefly, the build system will no longer be able to determine the module dependency information at the outset, before starting the compilation while the compiler may not have access to all the (up-to-date) BMIs (binary module interfaces) to perform the compilation. As a result, the compiler will have to query (i.e., *call back* into) the build system on encountering every import declaration in order to obtain an (up-to-date) BMI that it can use (and which the build system might still have to compile, potentially triggering a recursive chain of callbacks).

Note also that this does not appear to be a transition-only issue since it is not evident the end state of a modularization process should be a codebase without any non-module translation units. For example, it is not clear why the translation unit that defines `main()` would ever need to be a module. [Note: One such reason could be unit testing: `main()` may need to belong to a module in order to gain access to non-exported entities.]

As a result, we propose that rules similar to the module preamble be applied to the non-module translation units, with the exception for automatic mapping of `#include` directives to legacy header module imports. [Note: Such a mapping is specified as both optional and implementation-defined and so such a relaxation seems harmless; see [P1103R0 Section 2.3.3](#) Clause 2 for details].

BELLEVUE: Per the discussion at the meeting, there appears to be agreement that modularizing a codebase should eventually result in the replacement of all non-modular translation units with modules and that enforcing the preamble restrictions in such units would make the gradual modularization process difficult. [P1180R0](#) proposed an alternative approach which would have also resolved this issue. However, it was not adopted. As a result, we believe build system vendors may end up imposing additional ad hoc restrictions on non-modular translation units, such as that proposed in [P1180R0](#) or explicit specification of legacy module dependencies.

BELLEVUE: The issue identified by [P1180R0](#) with this proposal (header inclusions containing `imports`) also affects the global module fragment in modular translation units.

4 Preamble End

[P1103R0 Section 2.1](#) Clause 1 states:

"A module unit begins with a preamble, comprising a module declaration and a sequence of imports: [...] Within a module unit, imports may only appear within the preamble."

Furthermore, from [P1103R0 Section 19.3](#) Clause 1 it follows that the importation of modules within the preamble cannot depend on macros exported from legacy header units.

The motivation for these restriction is to allow tools (such as build systems) that wish to extract the module-related information from a module translation unit to parse the preamble without supplying any of the BMIs (binary module interfaces) for imported modules (legacy or not). It is expected that compiler implementations will provide support for preamble-only preprocessing that such tools will use (see, for example, GCC's `-fmodule-preamble`).

[P1103R0 Section 19.3](#) Clause 4 defines the preamble as a sequence of *preprocessing-tokens* that match a production pattern (*pp-preamble*). However, in practice, detecting where the preamble ends appears to be challenging since peeking at the next *preprocessing-token* may involve processing directives (such as `#include` or `#error`) that are difficult to do partially or undo. Consider this example:

```
module M;

import foo;
import "foo.h";
                                // (a)

#ifdef EXTRA
#  include "bar.h"
#endif
                                // (b)

void f ();
```

Where exactly the preamble ends in this example depends on whether `EXTRA` is a module-exported macro and what is inside header `bar.h`. Some possible scenarios:

1. If `EXTRA` is a module-exported macro, then the preamble ends at (a) unless `bar.h` contains import declarations in which case the translation unit is invalid.
2. If `EXTRA` is not a module-exported macro, then preamble ends at (a) unless `bar.h` contains import declarations in which case it ends at (b) (or somewhere inside `bar.h`, if it also contains other declarations).

However, without loading the BMI for legacy header module `fox.h` the compiler cannot know what kind of macro `EXTRA` is (or whether it is actually defined) and without preprocessing header `bar.h` it doesn't know what it contains. And speculatively preprocessing header `bar.h` may have various side effects. For example, it may contain `#error` or not even exist if `fox.h` does in fact define `EXTRA`.

To overcome this, the current (admittedly experimental) implementation in GCC suggests explicitly marking the preamble end with a stray semicolon. For example, if `bar.h` does exist, the user sees the following diagnostics:

```
m.cxx:6:1: warning: module preamble ended immediately before
           preprocessor directive
m.cxx:6:1: note: explicitly mark the end with an earlier ';'

```

However, if `bar.h` does not exist, GCC terminates with a fatal error before having a chance to issue the above suggestion.

Based on this we believe the current semantics of determining the preamble end will lead to brittle tooling with confusing diagnostics. As a result, we propose adding an explicit *preamble end marker* (or *preamble concluder*) similar to the leading module marker proposed in [P0713R1](#) and adopted by [P1103R0](#) (where it is called *module introducer*). For example:

```
module M;

import foo;
import "fox.h";

```

```
import;  // Preamble end.
```

```
#ifndef EXTRA
```

```
#  include "bar.h"
```

```
#endif
```

```
void f ();
```

Nobody will argue that this is inelegant but one way or another there appears to be a cost for supporting exportation of macros from modules.

BELLEVUE: There was no consensus at the meeting on whether to add the explicit preamble end marker. However, the following alternative syntax was generally viewed as a better option to either the stray semicolon or the empty `import:`

```
module M
{
    import foo;
    export import bar;
    import "fox.h";

} // Preamble end.

...
```

Or even (inspired by the Go's import declaration syntax):

```
module M;

import (
    foo,
    export bar,
    "fox.h"
); // Preamble end.

...
```

5 Module Partitions

This section contains a collection of notes on module partitions and their implications for tooling. At this stage it does not propose any changes to P1103R0.

Partition names are a sequence of identifiers, the same as the module names themselves ([P1103R0 Section 10.7.1](#)). It is not clear why this support for hierarchical partitions is desirable. On the other hand it will surely complicate the

mapping of the combined module/partition names to filesystem entities (for example, BMI files).

Implementation partitions can be imported by other translation units belonging to the same module ([P1103R0 Section 10.7.1](#) Clause 4, 10). In other words, we now have *importation of implementations* which means there will have to be BMIs for them. ~~The fact that there can (presumably) be both interface and implementation partition units for the same partition further complicates things (do they end up with separate BMIs or is it merged, etc). What happens during importation of such a dual-unit partition does not appear to be specified (presumably entities from both become visible).~~

At the Rapperswill meeting it was mentioned that various strategies are available to implementations when it comes to module partitions: they can be represented as separate partition BMIs or they can contribute to a combined module BMI. However, it feels that a combined BMI approach will reduce the build system's ability to parallelize compilation.

BELLEVUE: As discussed in [P1180R0](#), the above crossed-out understanding is incorrect: a module partition is a single (interface or implementation) translation unit. In other words, a partition is always an "interface", but depending on what kind of partition it is, it can be "public" (with its exported declarations visible outside of the module) or "private" (with all its declarations visible but only inside the module).

6 Acknowledgments

Thanks to Nathan Sidwell for clarifications on the preamble end detection algorithm as implemented in GCC. Thanks to Richard Smith for the response paper ([P1180R0](#)) as well as further clarifications on the module partition semantics.